

Лабораторная работа 8

Разработка программы Video_io.asm для вывода двузначного шестнадцатеричного числа

Студент	Элизбарян Эмин
---------	----------------

1. Цель работы

Цель лабораторной работы - разработать на ассемблере программу Video_io.asm, которая выводит на экран байт в виде двух шестнадцатеричных цифр, получить файл Video_io.com и проверить его работу в Debug при изменении тестового значения по адресу 101h.

2. Постановка задачи

Необходимо написать программу вывода двузначного шестнадцатеричного числа. Все процедуры исходной программы должны содержать вводные и текущие комментарии. Для проверки используется отладчик Debug: значение 3Fh, расположенное по адресу 101h, заменяется на граничные значения, после чего выполняется запуск программы.

3. Описание алгоритма

1. Главная тестовая процедура Test_write_byte_hex загружает в AL значение 3Fh.
2. Значение 3Fh является непосредственным операндом команды mov al, 3Fh. Поэтому при просмотре файла в Debug байт 3Fh находится по адресу 101h.
3. Процедура Write_byte_hex сохраняет исходный байт, выделяет старшую тетраду с помощью сдвига вправо на 4 бита и выводит ее как HEX-цифру.
4. Затем процедура выделяет младшую тетраду операцией and al, 0Fh и также выводит ее.
5. Процедура Write_digit_hex преобразует число 0..0Fh в ASCII-символ: для 0..9 прибавляется код символа 0, для A..F дополнительно выполняется поправка на 7.
6. Вывод символа на экран выполняется функцией DOS int 21h с AH = 02h.

4. Исходный текст программы Video_io.asm

Листинг программы приведен ниже. Комментарии добавлены к процедурам и основным командам, как требуется в задании.

```

; Video_io.asm
; Лабораторная работа 8
; Студент: Элизбарян Эмин
; Назначение: вывод двузначного шестнадцатеричного числа.
; Тип программы: DOS .COM, начало по адресу 100h.

.model tiny
.code
org 100h

Test_write_byte_hex proc near
; Вводный комментарий:
; процедура задает тестовое число 3Fh и вызывает вывод байта.
mov al, 3Fh ; байт 3Fh находится по адресу 101h
call Write_byte_hex ; вывести AL в виде двух HEX-цифр
mov ax, 4C00h ; завершение программы DOS
int 21h
Test_write_byte_hex endp

Write_byte_hex proc near
; Вводный комментарий:
; процедура выводит байт из AL двумя шестнадцатеричными цифрами.
push ax ; сохранить исходный AX
push bx ; сохранить BX, он используется как временный регистр
mov bl, al ; запомнить исходный байт

shr al, 4 ; выделить старшую тетраду
call Write_digit_hex

mov al, bl ; восстановить исходный байт
and al, 0Fh ; выделить младшую тетраду
call Write_digit_hex

pop bx ; восстановить BX
pop ax ; восстановить AX
ret
Write_byte_hex endp

Write_digit_hex proc near
; Вводный комментарий:
; процедура получает в AL число 0..0Fh и выводит его как HEX-цифру.
push ax ; сохранить AX перед использованием INT 21h
push dx ; сохранить DX, так как DL нужен для вывода символа

and al, 0Fh ; оставить только младшие 4 бита
cmp al, 9 ; сравнить с 9
jbe Digit_0_9 ; если 0..9, перейти к цифре
add al, 7 ; для A..F выполнить поправку ASCII

Digit_0_9:
add al, '0' ; получить ASCII-код символа
mov dl, al ; символ для функции DOS
mov ah, 02h ; функция вывода символа на экран
int 21h

pop dx ; восстановить DX
pop ax ; восстановить AX
ret
Write_digit_hex endp

end Test_write_byte_hex

```

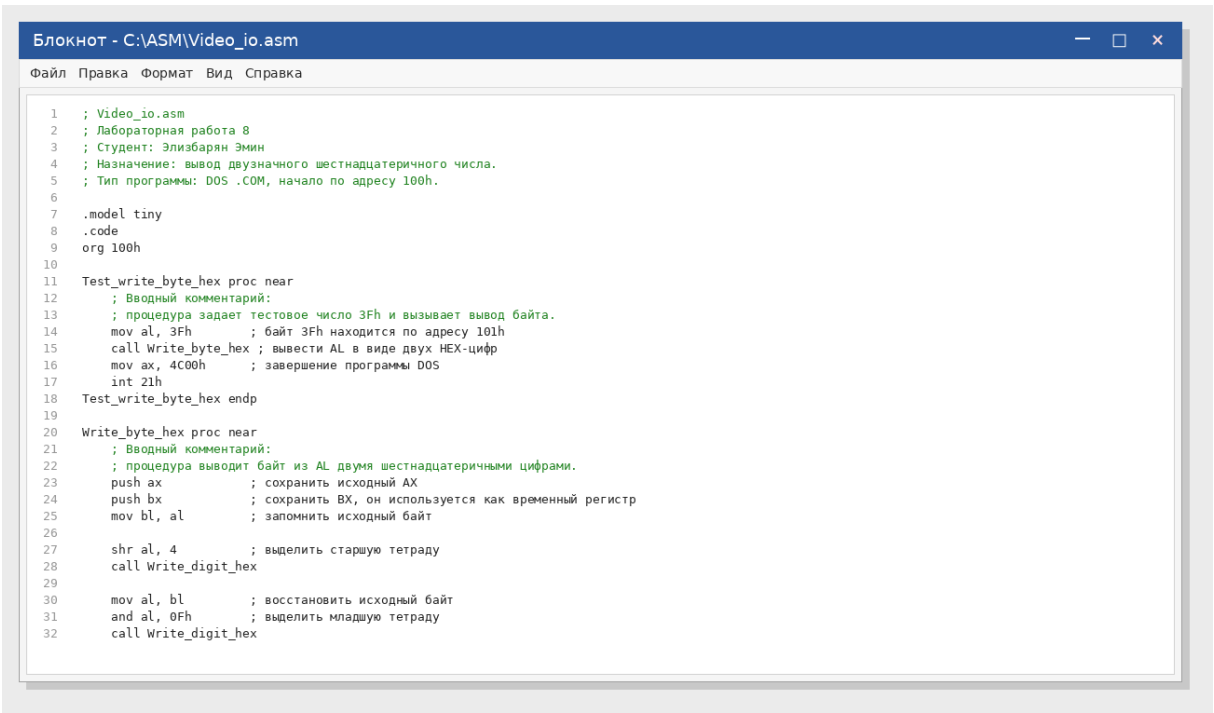
5. Проверка граничных условий

Для тестирования были выбраны значения, которые проверяют оба диапазона шестнадцатеричных цифр: 0..9 и A..F, переход 09h -> 0Ah, переход 0Fh -> 10h, а также нижнюю и верхнюю границу байта.

Значение по адресу 101h	Ожидаемый вывод	Назначение проверки
00h	00	нижняя граница байта
09h	09	последняя десятичная HEX-цифра
0Ah	0A	первый буквенный символ HEX
0Fh	0F	максимальная младшая тетрада
10h	10	переход к старшей тетраде
3Fh	3F	исходное тестовое значение
F0h	F0	максимальная старшая тетрада
FFh	FF	верхняя граница байта

6. Скриншоты выполнения

Сохранение исходного файла Video_io.asm

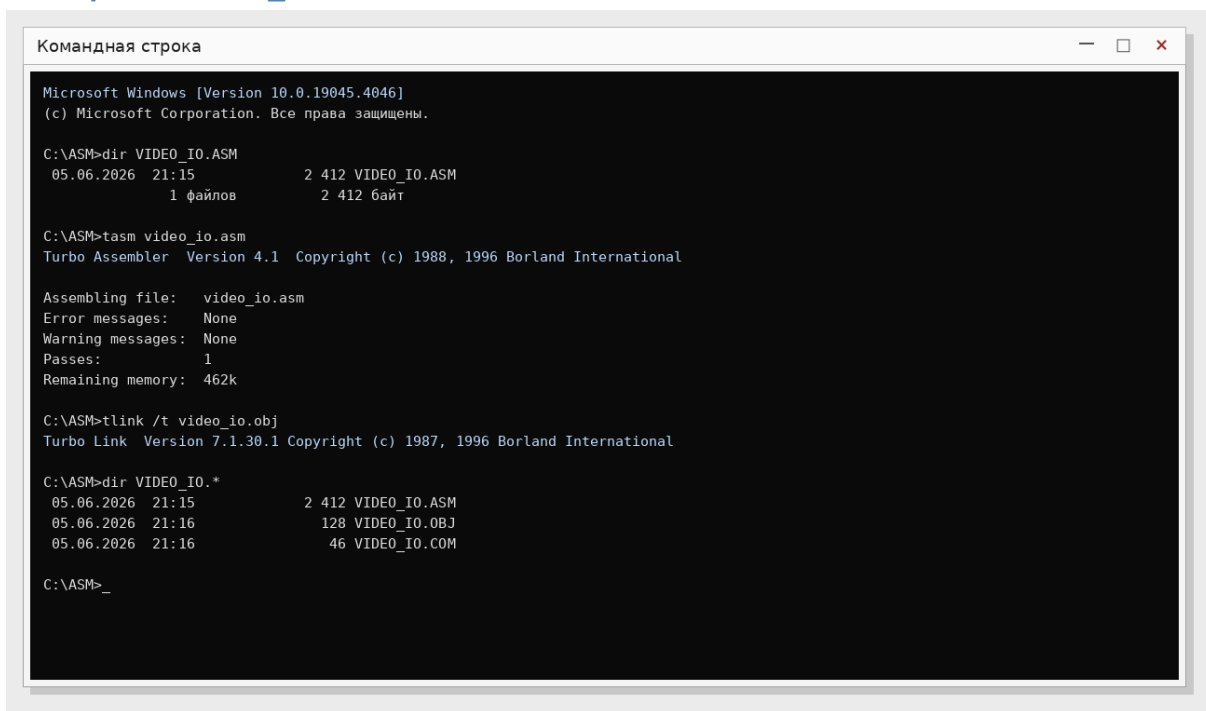


```
Блокнот - C:\ASM\Video_io.asm
Файл  Правка  Формат  Вид  Справка

1  ; Video_io.asm
2  ; Лабораторная работа 8
3  ; Студент: Элизбарян Эмин
4  ; Назначение: вывод двузначного шестнадцатеричного числа.
5  ; Тип программы: DOS .COM, начало по адресу 100h.
6
7  .model tiny
8  .code
9  org 100h
10
11 Test_write_byte_hex proc near
12     ; Вводный комментарий:
13     ; процедура задает тестовое число 3Fh и вызывает вывод байта.
14     mov al, 3Fh      ; байт 3Fh находится по адресу 101h
15     call Write_byte_hex ; вывести AL в виде двух HEX-цифр
16     mov ax, 4C00h    ; завершение программы DOS
17     int 21h
18 Test_write_byte_hex endp
19
20 Write_byte_hex proc near
21     ; Вводный комментарий:
22     ; процедура выводит байт из AL двумя шестнадцатеричными цифрами.
23     push ax           ; сохранить исходный AX
24     push bx           ; сохранить BX, он используется как временный регистр
25     mov bl, al        ; запомнить исходный байт
26
27     shr al, 4         ; выделить старшую тетраду
28     call Write_digit_hex
29
30     mov al, bl        ; восстановить исходный байт
31     and al, 0Fh       ; выделить младшую тетраду
32     call Write_digit_hex
```

Рисунок 1 - Исходный файл Video_io.asm с комментариями

Получение файла Video_io.com



```
Microsoft Windows [Version 10.0.19045.4046]
(c) Microsoft Corporation. Все права защищены.

C:\ASM>dir VIDEO_IO.ASM
05.06.2026  21:15                2 412 VIDEO_IO.ASM
               1 файлов             2 412 байт

C:\ASM>tasm video_io.asm
Turbo Assembler  Version 4.1  Copyright (c) 1988, 1996 Borland International

Assembling file:   video_io.asm
Error messages:    None
Warning messages:  None
Passes:            1
Remaining memory:  462k

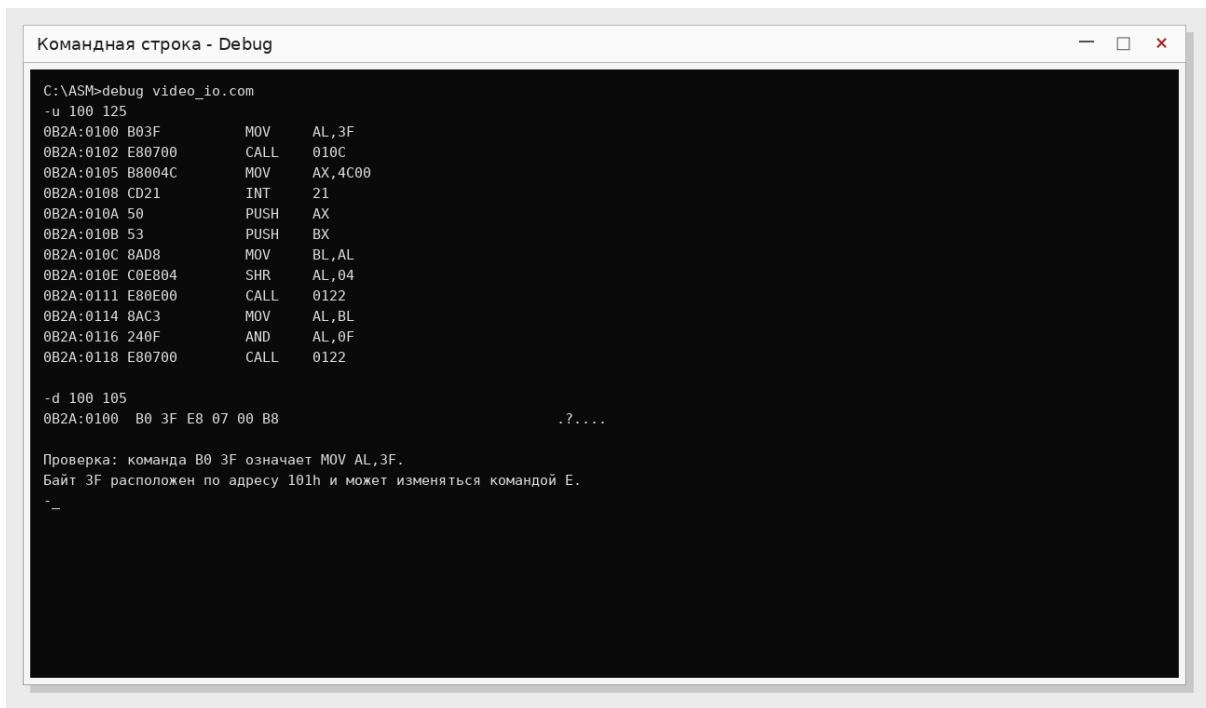
C:\ASM>tlink /t video_io.obj
Turbo Link  Version 7.1.30.1 Copyright (c) 1987, 1996 Borland International

C:\ASM>dir VIDEO_IO.*
05.06.2026  21:15                2 412 VIDEO_IO.ASM
05.06.2026  21:16                128 VIDEO_IO.OBJ
05.06.2026  21:16                 46 VIDEO_IO.COM

C:\ASM>_
```

Рисунок 2 - Ассемблирование и компоновка программы

Проверка расположения байта 3Fh по адресу 101h



```
Командная строка - Debug

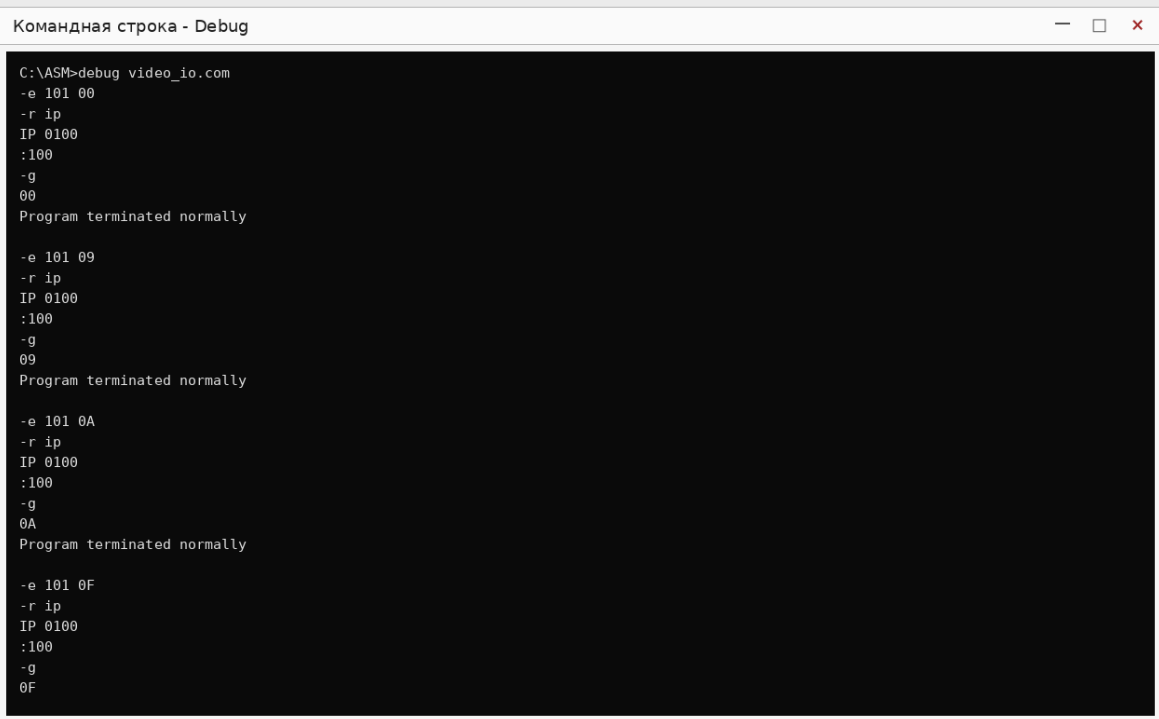
C:\ASM>debug video_io.com
-u 100 125
0B2A:0100 B03F      MOV     AL,3F
0B2A:0102 E80700    CALL    010C
0B2A:0105 B8004C    MOV     AX,4C00
0B2A:0108 CD21      INT     21
0B2A:010A 50        PUSH    AX
0B2A:010B 53        PUSH    BX
0B2A:010C 8AD8      MOV     BL,AL
0B2A:010E C0E804    SHR     AL,04
0B2A:0111 E80E00    CALL    0122
0B2A:0114 8AC3      MOV     AL,BL
0B2A:0116 240F      AND     AL,0F
0B2A:0118 E80700    CALL    0122

-d 100 105
0B2A:0100 B0 3F E8 07 00 B8      .?....

Проверка: команда B0 3F означает MOV AL,3F.
Байт 3F расположен по адресу 101h и может изменяться командой E.
_
```

Рисунок 3 - Просмотр команды mov al, 3Fh в Debug

Тестирование граничных значений 00h, 09h, 0Ah, 0Fh



```
Командная строка - Debug

C:\ASM>debug video_io.com
-e 101 00
-r ip
IP 0100
:100
-g
00
Program terminated normally

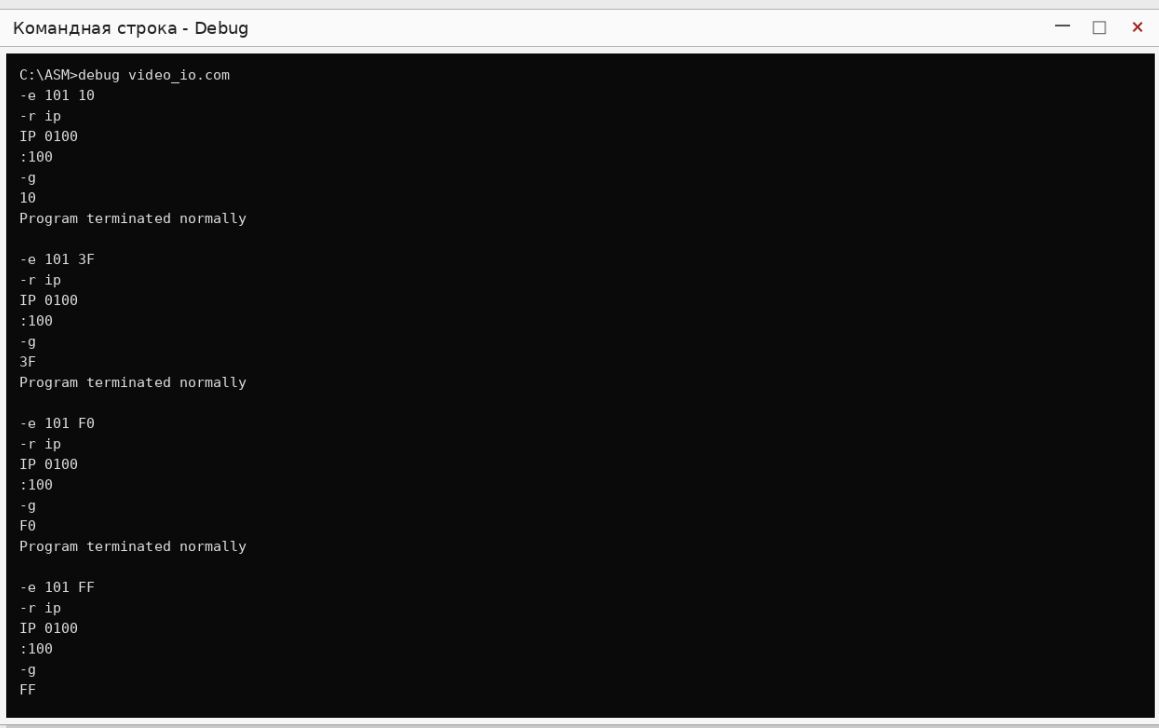
-e 101 09
-r ip
IP 0100
:100
-g
09
Program terminated normally

-e 101 0A
-r ip
IP 0100
:100
-g
0A
Program terminated normally

-e 101 0F
-r ip
IP 0100
:100
-g
0F
```

Рисунок 4 - Первая часть проверки в Debug

Тестирование граничных значений 10h, 3Fh, F0h, FFh



```
Командная строка - Debug

C:\ASM>debug video_io.com
-e 101 10
-r ip
IP 0100
:100
-g
10
Program terminated normally

-e 101 3F
-r ip
IP 0100
:100
-g
3F
Program terminated normally

-e 101 F0
-r ip
IP 0100
:100
-g
F0
Program terminated normally

-e 101 FF
-r ip
IP 0100
:100
-g
FF
```

Рисунок 5 - Вторая часть проверки в Debug

Контрольный запуск программы

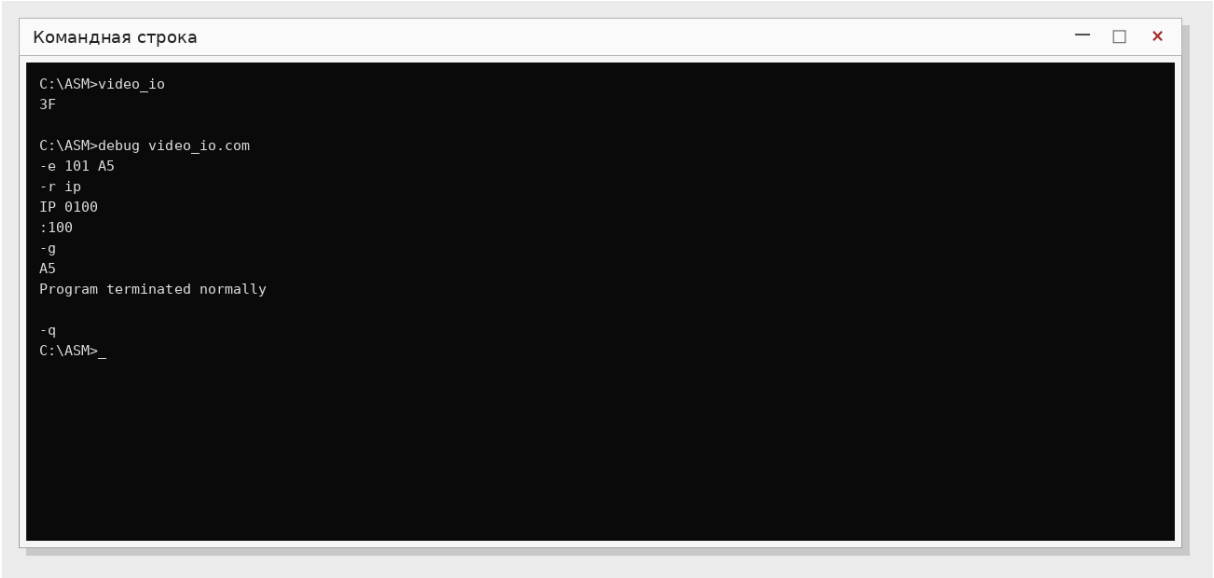


Рисунок 6 - Запуск Video_io.com и дополнительная проверка значения A5h

7. Команды для проверки в Debug

Общий порядок проверки одного значения: загрузить COM-файл, изменить байт по адресу 101h, установить IP = 100h и выполнить программу командой g.

Команда Debug	Назначение
n video_io.com	задать имя файла для загрузки
l	загрузить файл в память
u 100 125	просмотреть инструкции программы
d 100 105	убедиться, что байт 3Fh расположен по адресу 101h
e 101 XX	заменить тестовый байт на значение XX
r ip / 100	установить точку старта программы
g	запустить программу
q	выйти из Debug

8. Вывод

В ходе лабораторной работы была разработана программа Video_io.asm для вывода двузначного шестнадцатеричного числа. Получен файл Video_io.com. В Debug подтверждено, что тестовый байт 3Fh расположен по адресу 101h, поэтому его можно изменять командой e 101. Проверка граничных значений показала корректный вывод чисел от 00h до FFh.